

API-Football Ingestor

Operator Reference

Bronze layer commands, scheduler, and date-range backfills

1. Overview

The ingestor fetches data from API-Football into a Postgres *bronze layer* (raw, append-only). It runs on demand from your terminal and automatically once a day via a scheduler container.

Three containers make up the stack:

- **ingestor-postgres** — the database (always running on host port 5434)
- **ingestor-scheduler** — a small cron container that triggers the daily run at 07:00 Europe/Berlin
- **ingestor** — the actual ingestor (built on demand; profile `cli`)

All commands assume your working directory is

`C:\Users\Dumebi\Arbibet\api-football-ingestor`.

2. Stack lifecycle

Command	What it does
<code>docker compose up -d postgres</code>	Start just the database (always-on)
<code>docker compose --profile scheduler up -d scheduler</code>	Start the cron scheduler
<code>docker compose ps</code>	List running containers + their host ports
<code>docker compose down</code>	Stop everything (database files persist)
<code>docker compose down -v</code>	Stop everything AND delete the database. Use with care

3. Running the ingestor — daily / one-off

Command	What it does
<code>docker compose run --rm ingestor ingest</code>	Full ingest now: all leagues, current season, last 2 days of fixtures
<code>docker compose run --rm ingestor ingest --leagues 39,140</code>	Only Premier League + La Liga
<code>docker compose run --rm ingestor ingest --season 2024</code>	Override the season
<code>docker compose run --rm ingestor ingest --lookback-days 7</code>	Override the rolling window (default 2)
<code>docker compose run --rm ingestor ingest --resume <run-uuid></code>	Resume an interrupted run
<code>docker compose run --rm ingestor ingest --no-migrate</code>	Skip schema migration step

Command	What it does
<code>docker compose run --rm ingestor migrate</code>	Apply schema migrations only

Resuming an interrupted run

Pass the `run_id` from `ingestion_runs`. Work that already succeeded is skipped; failed or unfinished tasks are retried. The original flags are **not** reloaded, so repeat them (e.g. `--from/--to`, `--leagues`).
Example:

```
docker compose run --rm ingestor ingest --resume f0743395-eba9-4418-9772-25d1dc5c3da1
```

Look up the original flags for that run first:

```
SELECT status, args
FROM ingestion_runs
WHERE run_id = 'f0743395-eba9-4418-9772-25d1dc5c3da1';
```

4. Running the ingestor — historical backfills (NEW)

Use `--from` and `--to` (format DD-MM-YYYY) to pull fixture details and halftime statistics for a specific historical window. These flags override `--lookback-days`. Both dates are inclusive.

Command	What it does
<code>docker compose run --rm ingestor ingest --from 01-03-2025 --to 31-03-2025</code>	Backfill all fixtures in March 2025
<code>docker compose run --rm ingestor ingest --from 01-01-2025 --to 31-01-2025 --leagues 39</code>	Premier League only, January 2025
<code>docker compose run --rm ingestor ingest --from 15-04-2025</code>	From that date until now
<code>docker compose run --rm ingestor ingest --to 31-12-2024</code>	Everything up to end of 2024

Note: Phase A still fetches every league's fixtures as usual; the date filter only affects which fixtures get the rich details + halftime stats (Phase B).

5. Rebuilding after code changes

Command	What it does
<code>docker compose build ingestor</code>	Rebuild the ingestor image after Python code changes
<code>docker compose --profile scheduler up -d --force-recreate scheduler</code>	Restart scheduler so it picks up new .env values or compose changes

6. Inspecting & debugging

Command	What it does
<code>docker compose logs -f postgres</code>	Follow Postgres logs
<code>docker exec ingestor-scheduler crontab -l</code>	Show the active cron schedule
<code>docker exec ingestor-scheduler date</code>	Confirm scheduler timezone (should show CEST/CET)
<code>docker exec ingestor-scheduler tail -f /var/log/ingest.log</code>	Watch daily run logs in real time
<code>docker exec ingestor-scheduler tail -n 500 /var/log/ingest.log</code>	Last 500 log lines
<code>docker exec -it ingestor-postgres psql -U ingestor -d ingestor</code>	Open psql shell inside Postgres

7. Manual scheduler trigger

Trigger the same run the cron will fire at 07:00, immediately:

```
docker exec ingestor-scheduler sh -c "docker compose -f /workspace/docker-compose.yml run --rm ingestor ingest"
```

8. Useful SQL snippets

Run these in DBeaver (Host=localhost, Port=5434, DB=ingestor, User=ingestor, Password=ingestor) or via psql.

Most recent runs

```
SELECT run_id, started_at, finished_at, status
FROM ingestion_runs
ORDER BY started_at DESC
LIMIT 10;
```

Dead-letter summary for the latest run

```
SELECT endpoint, error_class, count(*)
FROM dead_letter
WHERE run_id = (SELECT run_id FROM ingestion_runs
ORDER BY started_at DESC LIMIT 1)
GROUP BY 1, 2
ORDER BY 3 DESC;
```

Row counts per bronze table

```
SELECT 'bronze_leagues' AS table, count(*) FROM bronze_leagues
UNION ALL SELECT 'bronze_fixtures', count(*) FROM bronze_fixtures
UNION ALL SELECT 'bronze_fixture_details', count(*) FROM bronze_fixture_details
UNION ALL SELECT 'bronze_halftime_stats', count(*) FROM bronze_halftime_stats;
```

Find fixtures missing detail records (gaps)

```
SELECT f.fixture_id
FROM (SELECT DISTINCT (payload->'response'->0->'fixture'->'id')::bigint AS fixture_id
FROM bronze_fixtures) f
LEFT JOIN bronze_fixture_details d ON d.fixture_id = f.fixture_id
WHERE d.fixture_id IS NULL
LIMIT 50;
```

9. Configuration knobs (.env)

Edit `C:\Users\Dumebi\Arbibet\api-football-ingestor\.env` then either rebuild (for code-affecting settings) or just rerun (for runtime settings).

Setting	Default	What it controls
RATE_LIMIT_PER_MIN	240	Calls per minute (kept under the 300 plan cap as safety margin)
RATE_LIMIT_WINDOW_SEC	65	Sliding window length
DAILY_QUOTA	7500	Daily call cap. Must match the plan (Pro = 7,500; check /status)
THREAD_POOL_SIZE	8	Parallel workers
LOOKBACK_DAYS	2	Default rolling Phase-B window (now 2 for daily runs)
HTTP_RETRY_MAX_ATTEMPTS	7	Max retries per failed call
TZ	Europe/Berlin	Scheduler timezone
POSTGRES_HOST_PORT	5434	Host port for DBeaver

10. Ingestion failure report

`docs\ingestion-failure-report.pdf` summarises every run: outcome by day, API calls against the daily limit, dead letters by cause, and days with no scheduled run. The builder reads the live database, so regenerate the report after any bad night to get current numbers. Run these from PowerShell on the host (not inside Docker). Postgres must be up.

Command	What it does
<code>.venv\Scripts\python -m pip install reportlab matplotlib tzdata</code>	One-time: install the PDF and chart libraries into the project venv
<code>docker compose up -d postgres</code>	Make sure the database is reachable on <code>POSTGRES_HOST_PORT</code>
<code>.venv\Scripts\python docs_build_failure_report.py</code>	Rebuild <code>docs\ingestion-failure-report.pdf</code> from live data
<code>.venv\Scripts\python docs_build_pdf.py</code>	Rebuild this manual (<code>docs\ingestor-commands.pdf</code>)
<code>start docs\ingestion-failure-report.pdf</code>	Open the report in the default PDF viewer

Before recreating the scheduler, save its log. The root cause of a crashed run (e.g. the daily-limit message) is only in `/var/log/ingest.log` inside the scheduler container. `--force-recreate` wipes it.

```
docker cp ingestor-scheduler:/var/log/ingest.log ingest-backup.log
```

Why did a run crash?

```
docker exec ingestor-scheduler sh -c "grep run.crashed /var/log/ingest.log | tail -n 3"
```

Plan, expiry and today's usage (uses no quota)

```
docker exec ingestor-scheduler sh -c 'wget -qO- --header="x-rapidapi-key: $APIFOOTBALL_KEY" --header="x-rapidapi-host: v3.football.api-sports.io" https://v3.football.api-sports.io/status'
```

11. Glossary

Term	Plain-English meaning
Bronze layer	The raw, untouched copy of API responses. Append-only
Phase A / Phase B	A = fetch leagues + their fixtures. B = for each fixture, fetch rich details + halftime stats
Rate limit	Max calls per minute the API allows
Quota	Bigger daily ceiling
Dead letter	Table of permanently-failed calls — needs human attention
Retryable / non-retryable	Temporary problems are retried; permanent ones aren't
Retry-After	A header telling us exactly how long to wait
Idempotent	Running twice gives the same result as once — safe to resume
Cron	A scheduler. "0 7 * * *" = every day at 07:00
DSN	Database connection string
Hash dedup	Fingerprint the response, skip storing if it matches last fingerprint